



FACULTAD
DE CIENCIAS
ECONÓMICAS

Econometría II

Primer semestre 2025

Trabajo práctico grupal N° 1

Cátedra: Moneta Pizarro

Integrantes grupo 6:

ARIAS, Maria Pia (45348034)

FERIOZZI, Candela (45091097)

LOSANO, Julieta (45405384)

ROMERO PAVANI, Francisco (42358355)

UNIDAD 1:

Realizar las siguientes operaciones con el código más eficiente posible de R:

- a) Crear un vector "x" que contenga una secuencia de números enteros. Esta secuencia debe comenzar en 15 e incrementar en 3 hasta alcanzar 54, luego decrecer en 2 hasta volver a 15. Mostrar el resultado.
- b) Determinar la cantidad de elementos en "x" que son menores a 25.
- c) Calcular la suma de los primeros cuatro elementos de "x", donde cada elemento se pondera por su posición en el vector (es decir, el primer elemento se multiplica por 1, el segundo por 2, etc.).
- d) Extraer los elementos de "x" cuyas posiciones correspondan a números mayores a 30.
- e) Crear un vector "y" con los elementos 2, 9, 0, 2, 7, 4 y 0, y un vector "z" donde cada elemento sea el correspondiente elemento de "y" más su posición en "y" menos 3.
- f) Dividir cada elemento de "y" por su correspondiente de "z". Al resultado almacenarlo en un vector "u".
- g) Extraer todos los elementos de "u" que sean menores a los elementos correspondientes de "z". ¿Se obtiene lo esperado?
- h) Remover todos los objetos de la memoria. Luego, crear un vector "x" con los datos 2, 5, 3, un vector "y" con los datos 6, 4, 9 y un vector "z" con los datos 2, 2, 7.
- i) Armar una matriz de nombre "A" cuyos vectores columna sean "x", "y" y "z", en este orden.
- j) Obtener la dimensión (orden) de "A".
- k) Obtener la traspuesta y el determinante de "A".
- l) Verificar que el producto matricial entre "A" y su inversa es igual a la matriz identidad. ¿Qué tiene de extraño el resultado?
- m) Multiplicar la tercera fila de la matriz "A" por la inversa de A, empleando los distintos tipos de productos (punto a punto, matricial y de Kronecker).
- n) Opcional: resolver todo lo anterior con Python.

#UNIDAD 1= TRABAJO PRÁCTICO 1

```
#a)
x <- c(seq(15, 54, by=3), seq(52, 15, by=-2));x

#b)
sum(x < 25)
```

```

#c)
((x[1:4])*(1:4))

#d)
length(x)
x[30:33]

#e)
y <- c(2, 9, 0, 2, 7, 4, 0);y
z <- c((y+(1:7))-3);z

#f)
u <- c(y/z)
u

#g)
u[u<z]
#si, obtenemos los resultados esperados. Asi mismo observamos que el
segundo elemento que arroja la salida es clasificado como NA, ya que,
como el tercer elemento de "u" es un NaN y el tercer elemento de "z" es
un número, no puede efectuar la comparación.

#h)
x <- c(2, 5, 3);x
y <- c(6, 4, 9);y
z <- c(2, 2, 7);z

#i)
A <- cbind(x, y, z); A

#j)
dim(A)

#k)
t(A)
det(A)

#l)
A %*% solve(A)
#los triángulos superior e inferior que dan cero, algunos tienden a cero
aunque no lo sean de forma redonda.

#m)
A[,3] * solve(A)
A[,3] %*% solve(A)
A[,3] %x% solve(A)

POST CORRER
> #UNIDAD 1= TRABAJO PRÁCTICO 1
> #a)
> x <- c(seq(15, 54, by=3), seq(52, 15, by=-2));x
[1] 15 18 21 24 27 30 33 36 39 42 45 48 51 54 52 50 48 46 44 42 40 38

```

```

36 34 32 30 28 26 24 22
[31] 20 18 16
>
> #b)
> sum(x < 25)
[1] 9
>
> #c)
> ((x[1:4])*(1:4))
[1] 15 36 63 96
>
> #d)
> length(x)
[1] 33
> x[30:33]
[1] 22 20 18 16
>
> #e)
> y <- c(2, 9, 0, 2, 7, 4, 0);y
[1] 2 9 0 2 7 4 0
> z <- c((y+(1:7))-3);z
[1] 0 8 0 3 9 7 4
>
> #f)
> u <- c(y/z)
> u
[1]      Inf 1.1250000      NaN 0.6666667 0.7777778 0.5714286 0.0000000
>
> #g)
> u[u<z]
[1] 1.1250000      NA 0.6666667 0.7777778 0.5714286 0.0000000
> #si, obtenemos los resultados esperados. Asi mismo observamos que el
segundo elemento que arroja la salida es clasificado como NA, ya que,
como el tercer elemento de "u" es un NaN y el tercer elemento de "z" es
un numero, no puede efectuar la comparación.

> #h)
> x <- c(2, 5, 3);x
[1] 2 5 3
> y <- c(6, 4, 9);y
[1] 6 4 9
> z <- c(2, 2, 7);z
[1] 2 2 7
> #i)
> A <- cbind(x, y, z); A
      x y z

```

```

[1,] 2 6 2
[2,] 5 4 2
[3,] 3 9 7
> #j)
> dim(A)
[1] 3 3
> #k)
> t(A)
  [,1] [,2] [,3]
x     2     5     3
y     6     4     9
z     2     2     7
> det(A)
[1] -88
> #l)
> A %*% solve(A)
      [,1]      [,2] [,3]
[1,] 1.000000e+00  6.938894e-17  0
[2,] 0.000000e+00  1.000000e+00  0
[3,] 4.440892e-16 -3.469447e-17  1
> #los triangulos superior e inferior que dan cero, algunos tienden a
cero aunque no lo sean de forma redonda.
> #m)
> A[,3] * solve(A)
      [,1]      [,2]      [,3]
x -0.2272727  5.454545e-01 -0.09090909
y  0.6590909 -1.818182e-01 -0.13636364
z -2.6250000 -1.457168e-16  1.75000000
> A[,3] %*% solve(A)
      [,1]      [,2]      [,3]
[1,] -2.193182  0.3636364  1.522727
> A[,3] %x% solve(A)
      [,1]      [,2]      [,3]
[1,] -0.2272727  5.454545e-01 -0.09090909
[2,]  0.6590909 -1.818182e-01 -0.13636364
[3,] -0.7500000 -4.163336e-17  0.50000000
[4,] -0.2272727  5.454545e-01 -0.09090909
[5,]  0.6590909 -1.818182e-01 -0.13636364
[6,] -0.7500000 -4.163336e-17  0.50000000
[7,] -0.7954545  1.909091e+00 -0.31818182
[8,]  2.3068182 -6.363636e-01 -0.47727273
[9,] -2.6250000 -1.457168e-16  1.75000000

```

EXTRA: TODO EL PYTHON

```
#inciso a
import numpy as np
dir(np)
x = np.concatenate((np.arange(15, 54, 3), np.arange(54, 15, -2)));x
#inciso b
np.sum(x < 25)
#inciso c
x[:4] * np.array([1, 2, 3, 4])
#inciso d
x[29:33]
#ojo aca que python NO incluye el valor from
#inciso e
y = np.array([2, 9, 0, 2, 7, 4, 0]); y
z = np.concatenate((y, np.arange(1, 7) - 3)); z
#inciso f
u = y / z ; u
#no se puede dividir, no tienen la misma longitud, se puede solucionar el
problema modificando la longitud de z, debido a que Python
#inciso g
#no se puede realizar debido a que no pudimos obtener u
RESULTADOS
#inciso a
>>> x = np.concatenate((np.arange(15, 54, 3), np.arange(54, 15, -2)));x
array([15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48, 51, 54, 52, 50, 48,
       46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16])
>>> #inciso b
>>> np.sum(x < 25)
np.int64(9)
>>> #inciso c
>>> x[:4] * np.array([1, 2, 3, 4])
array([15, 36, 63, 96])
>>> #inciso d
>>> x[29:33]
array([22, 20, 18, 16])
>>> #ojo aca que python NO incluye el valor from
>>> #inciso e
>>> y = np.array([2, 9, 0, 2, 7, 4, 0]); y
array([2, 9, 0, 2, 7, 4, 0])
>>> z = np.concatenate((y, np.arange(1, 7) - 3)); z
array([ 2,  9,  0,  2,  7,  4,  0, -2, -1,  0,  1,  2,  3])
>>> #inciso f
>>> u = y / z ; u
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ValueError: operands could not be broadcast together with shapes (7,) (13,)
>>> #no se puede dividir, no tienen la misma longitud, se puede solucionar el
problema modificando la longitud de z, debido a que Python
```

```
>>> #inciso g
>>> #no se puede realizar debido a que no pudimos obtener u
```

UNIDAD 2:

Parte I: Análisis estadístico y econométrico básico

Ejercicio 1: Utilizando la base de datos disponible en el data frame CPS1988 del paquete AER, que también fue utilizado en el video de la clase de regresión lineal múltiple:

a) Obtener un resumen de las principales medidas de posición de todas las variables e interpretar los resultados.

```
summary(CPS1988)
```

```
> summary(CPS1988)
   wage      education      experience      ethnicity      smsa
Min.   :  50.05   Min.   :  0.00   Min.   : -4.0   Length:28155   Length:28155
1st Qu.: 308.64   1st Qu.:12.00   1st Qu.:  8.0   Class :character   Class:character
Median : 522.32   Median :12.00   Median :16.0   Mode  :character   Mode  :character
Mean   : 603.73   Mean   :13.07   Mean   :18.2
3rd Qu.: 783.48   3rd Qu.:15.00   3rd Qu.:27.0
Max.   :18777.20   Max.   :18.00   Max.   :63.0

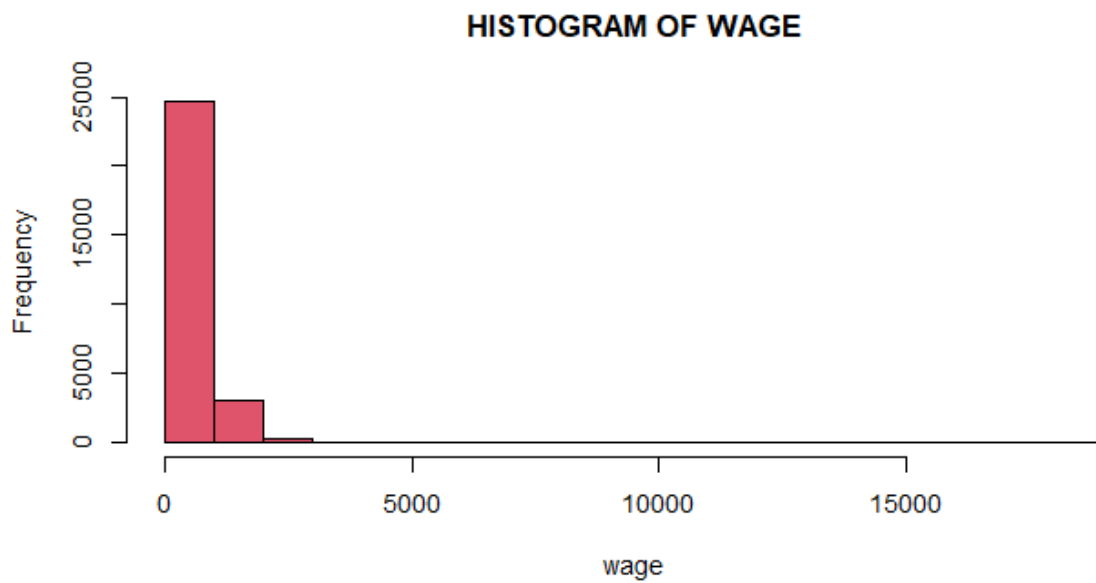
   region      parttime
Length:28155   Length:28155
Class :character   Class :character
Mode  :character   Mode  :character
```

Se calcularon medidas de posición como media, mediana, mínimos y máximos. Los resultados indican que:

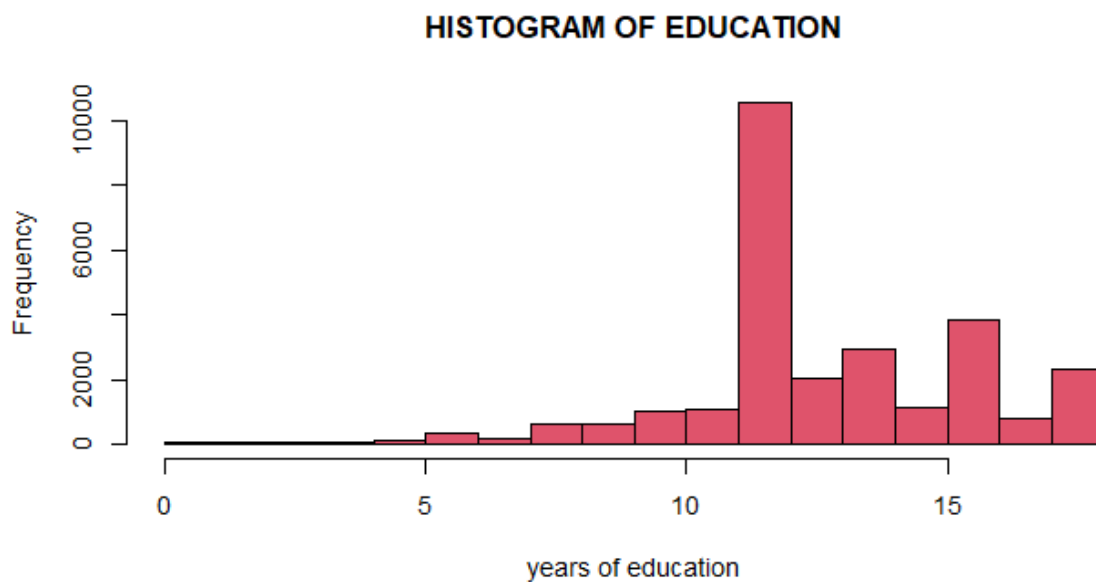
- **El salario (wage)** presenta gran dispersión: el salario mínimo es 50.05, pero el máximo alcanza los 18.777,20, lo que sugiere una distribución asimétrica y con outliers.
- **La educación (education)** tiene una media de 13 años y mediana de 12, indicando que la mayoría completó la secundaria.
- **La experiencia (experience)** muestra un promedio de 18.2 años, con casos incluso negativos, lo que probablemente sea error de carga o codificación.
- Las variables **ethnicity**, **smsa**, **region** y **parttime** son cualitativas (factores) y se analizaron en gráficos posteriores.

b) Elaborar gráficos y analizar las distribuciones univariadas de las variables.

```
hist(CPS1988$wage, col=2, main= "HISTOGRAM OF WAGE", xlab="wage")
```



```
hist(CPS1988$education, col=2, main= "HISTOGRAM OF EDUCATION",  
xlab="years of education")
```



```
hist(CPS1988$experience, col=2, main= "HISTOGRAM OF EXPERIENCE",  
xlab="years of experience")
```




Los histogramas de wage, education y experience muestran:

- Una distribución sesgada a la derecha para wage, confirmando la presencia de altos salarios atípicos.
- Distribuciones más simétricas en education y experience, aunque discretas (años enteros).

#despues de utilizar multiples codigos, vimos que la siguiente forma de graficar no es la mas eficiente en cuanto a economizar lineas, pero si no la haciamos asi, quedaban las etiquetas de los graficos al revés y no se veía a simple vista una clara interpretacion. Para compensar el uso excesivo de lineas, usamos la funcion con la cual se pueden exponer los 4 graficos juntos, a continuacion los pertinentes codigos:

#Grafico de torta en 3D

```
install.packages("plotrix")
```

```
library(plotrix)
```

#limpiamos la variable region, es decir, Sacamos espacios adelante y atrás de los valores de texto

```
CPS1988$region <- trimws(CPS1988$region)
```

Dividimos la pantalla en 2x2

```
par(mfrow = c(2, 2))
```

Gráfico 1: ethnicity

```
eth_freq <- table(CPS1988$ethnicity)
```

```

eth_pct <- round(eth_freq / sum(eth_freq) * 100)
eth_labels <- paste(names(eth_freq), ": ", eth_pct, "%", sep = "")
pie3D(eth_freq, labels = eth_labels, col = c("steelblue", "tomato"),
      explode = 0.1, main = "Etnicidad", labelcex = 1.1)

# Gráfico 2: smsa
smsa_freq <- table(CPS1988$smsa)
smsa_pct <- round(smsa_freq / sum(smsa_freq) * 100)
smsa_labels <- paste(names(smsa_freq), ": ", smsa_pct, "%", sep = "")
pie3D(smsa_freq, labels = smsa_labels, col = c("forestgreen", "orange"),
      explode = 0.1, main = "Zona Metropolitana (SMSA)", labelcex = 1.1)

# Gráfico 3: region (ya limpia)
region_freq <- table(CPS1988$region)
region_pct <- round(region_freq / sum(region_freq) * 100)
region_labels <- paste(names(region_freq), ": ", region_pct, "%", sep =
"")
pie3D(region_freq, labels = region_labels,
      col = c("darkred", "steelblue", "gold", "darkolivegreen"),
      explode = 0.1, main = "Región geográfica", labelcex = 1.1)

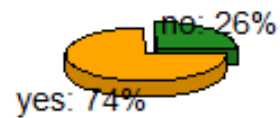
# Gráfico 4: parttime
pt_freq <- table(CPS1988$parttime)
pt_pct <- round(pt_freq / sum(pt_freq) * 100)
pt_labels <- paste(names(pt_freq), ": ", pt_pct, "%", sep = "")
pie3D(pt_freq, labels = pt_labels, col = c("deepskyblue", "salmon"),
      explode = 0.1, main = "Tipo de jornada (Part-time)", labelcex =
1.1)

```

Etnicidad



Zona Metropolitana (SMSA)



Región geográfica



Tipo de jornada (Part-time)

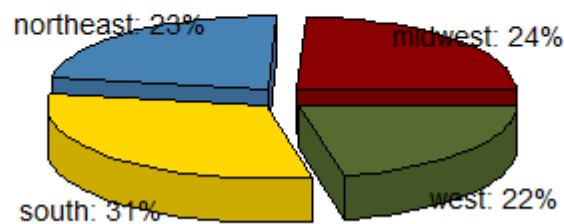


Lo que podemos interpretar de los respectivos gráficos es:

- **Etnicidad:** La muestra está dominada por personas caucásicas (92%), lo que puede limitar el análisis de diferencias étnicas.
- **Zona Metropolitana (SMSA):** El 74% vive en áreas urbanas, lo que sugiere un fuerte sesgo hacia contextos metropolitanos, con posibles efectos sobre los ingresos.
- **Región geográfica:** El Sur es la región más representada (31%), lo cual es relevante dado su contexto económico particular.
- **Tipo de jornada:** La mayoría trabaja full-time (91%), lo que puede influir en los niveles promedio de ingreso observados.

Haciendo un zoom a la distribución por región (debido a que no se alcanza a divisar con claridad en el anterior gráfico):

Distribución por región (3D)

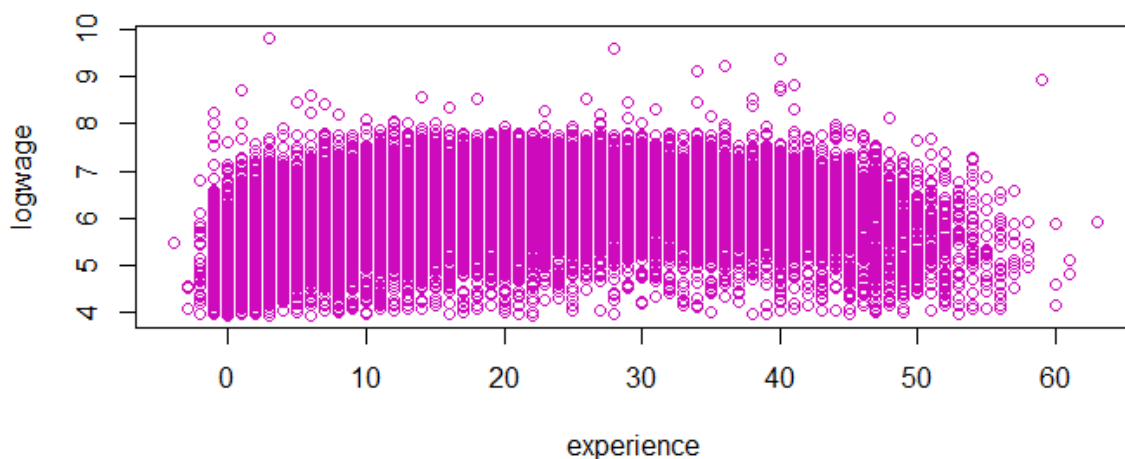


c) Construir diagramas de dispersión del logaritmo de los salarios reales, $\log(\text{wage})$, versus experiencia (experience) y educación (education). Interpretar los resultados.

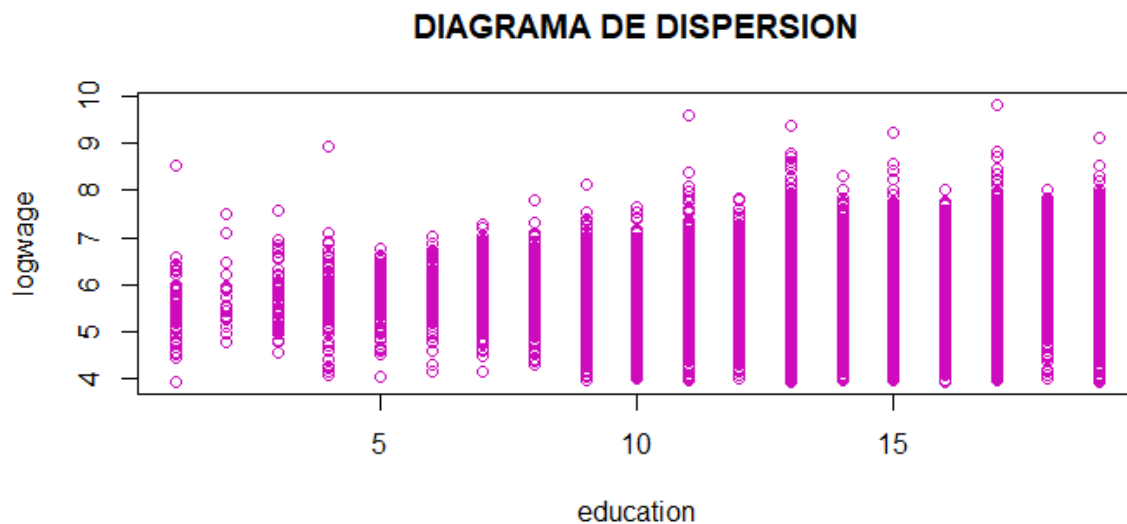
```
install.packages("AER")  
library(AER)
```

```
plot(log(CPS1988$wage)~(CPS1988$experience), data=CPS1988, col=6,  
main="DIAGRAMA DE DISPERSION", xlab="experience", ylab="logwage")
```

DIAGRAMA DE DISPERSION



```
plot(log(CPS1988$wage)~(CPS1988$education), data=CPS1988, col=6,  
main="DIAGRAMA DE DISPERSION", xlab="education", ylab="logwage")
```



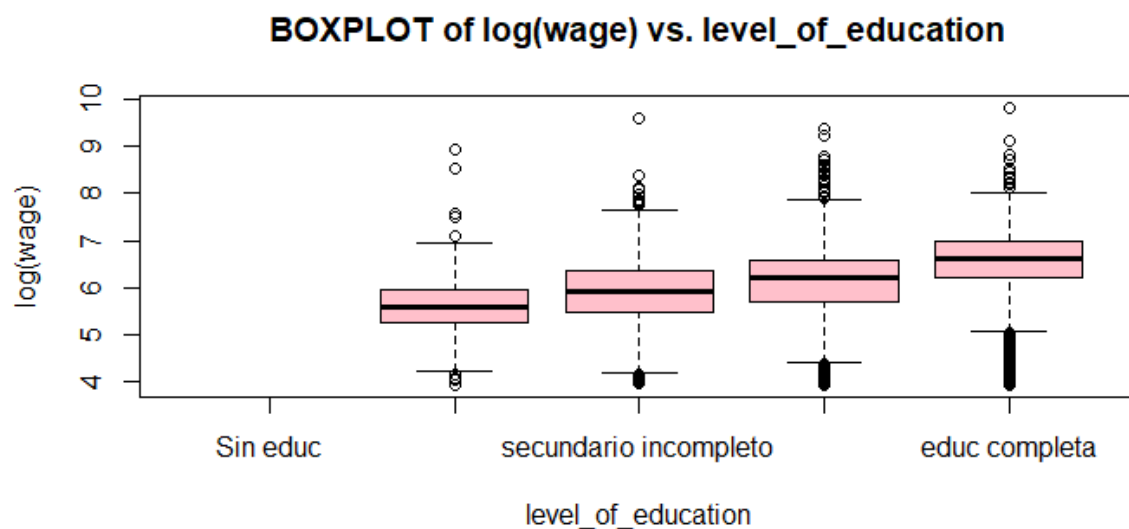
Se observa:

- Relación **positiva entre educación y log(salario)**: más años de educación se asocian a mayores salarios.
- Relación **más débil y no lineal con la experiencia**: los salarios aumentan con la experiencia pero tienden a estancarse o incluso disminuir en niveles muy altos.

d) Considerando que la educación (education) es una variable que toma pocos valores porque está computada como el número de años de escolarización, transformar sus datos en un factor (categorizar) y graficar diagramas de caja y brazos (boxplots) para los salarios en logaritmos, $\log(\text{wage})$, de cada nivel educativo. Extraer alguna conclusión.

```
CPS1988$level_of_education <- cut(CPS1988$education, breaks=c(-1, 0, 5,
12, 16, Inf), labels=c("Sin educ", "primaria incompleta", "secundario
incompleto", "universitario incompleto", "educ completa"), right=TRUE)
```

```
boxplot(log(CPS1988$wage) ~ CPS1988$level_of_education,
  data= CPS1988,
  col= "pink",
  main="BOXPLOT of log(wage) vs. level_of_education",
  ylab="log(wage)",
  xlab="level_of_education")
```

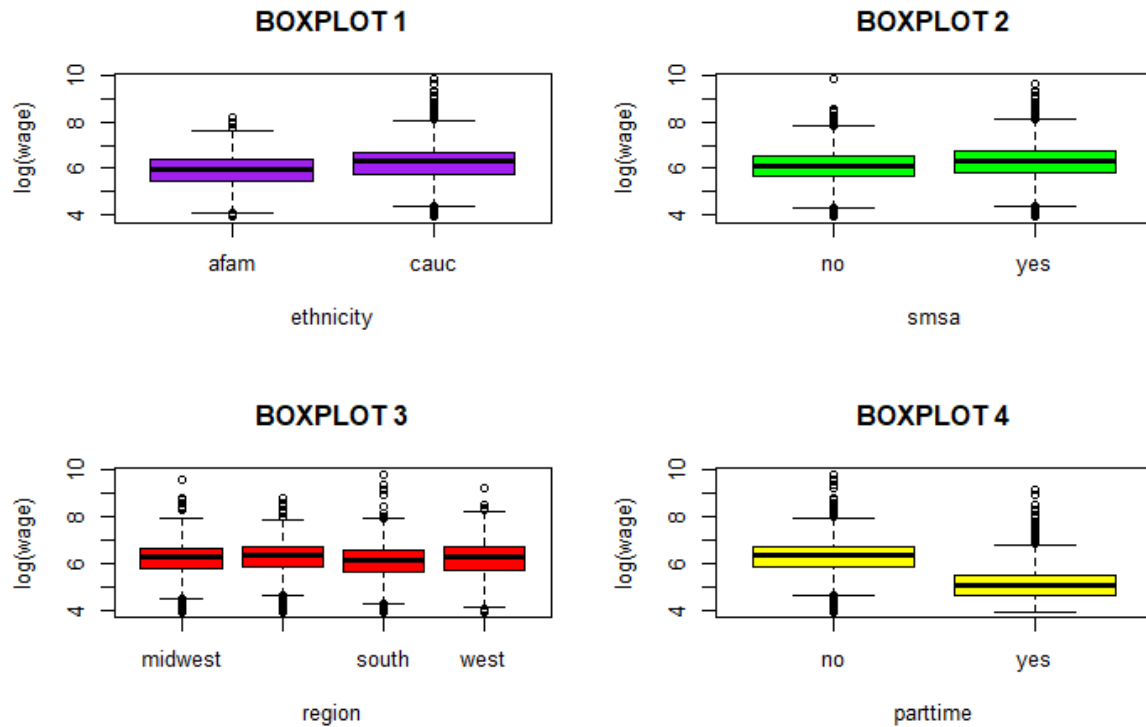


Se observa que:

- A mayor nivel educativo, **mayor salario esperado** y **menor dispersión relativa**.
- El aumento de ingresos es especialmente notorio en quienes completaron la universidad.

e) Los datos también contienen cuatro factores adicionales: ethnicity, smsa, region y parttime. Obtener gráficos adecuados para representar el logaritmo de los salarios, log(wage), versus cada uno de estos factores. Analizar e interpretar los resultados.

```
par(mfrow=c(2, 2))
boxplot(log(CPS1988$wage) ~ CPS1988$ethnicity, data= CPS1988,
main="BOXPLOT 1", ylab="log(wage)", xlab="ethnicity", col="purple")
boxplot(log(CPS1988$wage) ~ CPS1988$smsa, data= CPS1988, main="BOXPLOT
2", ylab="log(wage)", xlab="smsa", col="green")
boxplot(log(CPS1988$wage) ~ CPS1988$region, data= CPS1988, main="BOXPLOT
3", ylab="log(wage)", xlab="region", col="red")
boxplot(log(CPS1988$wage) ~ CPS1988$parttime, data= CPS1988,
main="BOXPLOT 4", ylab="log(wage)", xlab="parttime", col="yellow")
```



Los gráficos muestran que:

- Los trabajadores caucásicos ganan en promedio más que los afroamericanos.
- Quienes viven en áreas metropolitanas (smsa=yes) tienen salarios más altos.
- Quienes trabajan a tiempo parcial (parttime=yes) tienen salarios considerablemente menores.
- Hay diferencias regionales, aunque menos marcadas.

f) Estimar los retornos de los diferentes niveles educativos sobre los salarios en logaritmos, utilizando el resto de las variables como controles para evitar posibles sesgos. Interpretar cuidadosamente los resultados.

```
jour_lm <- lm((log(CPS1988$wage)) ~
CPS1988$level_of_education+CPS1988$ethnicity+CPS1988$smsa+CPS1988$region
+CPS1988$parttime, data=CPS1988)
jour_lm
```

```
> jour_lm <- lm((log(CPS1988$wage)) ~
CPS1988$level_of_education+CPS1988$ethnicity+CPS1988$smsa+CPS1988$region+CPS1988
$parttime, data=CPS1988)
> jour_lm
```

Call:

```
lm(formula = (log(CPS1988$wage)) ~ CPS1988$level_of_education +
  CPS1988$ethnicity + CPS1988$msa + CPS1988$region + CPS1988$parttime,
  data = CPS1988)
```

Coefficients:

```

                                (Intercept)
                                5.468148
CPS1988$level_of_educationsecundario incompleto
                                0.241814
CPS1988$level_of_educationuniversitario incompleto
                                0.440014
CPS1988$level_of_educationeduc completa
                                0.826137
CPS1988$ethnicitycauc
                                0.216590
CPS1988$msayes
                                0.144631
CPS1988$regionnortheast
                                0.049328
CPS1988$regionsouth
                                -0.057876
CPS1988$regionwest
                                0.006844
CPS1988$parttimeyes
                                -1.130727
```

Se estimó un modelo de regresión lineal múltiple con $\log(\text{wage})$ como dependiente, y como regresores: nivel educativo (factor), etnia, smsa, región y tipo de jornada.

Resultados relevantes:

- A medida que aumenta el nivel educativo, los retornos en salario aumentan significativamente. Por ejemplo, quienes tienen educación completa ganan un 82,6% más que quienes no tienen educación.
- Ser caucásico está asociado a un salario 21,7% más alto en promedio que ser afroamericano, ceteris paribus.
- Vivir en una zona metropolitana (smsa = yes) aumenta el salario en 14,5%.
- Trabajar a tiempo parcial implica una reducción salarial muy significativa, de -113% en $\log(\text{wage})$, lo cual es esperable por menor carga horaria.
- Las variables regionales muestran efectos menores.

Conclusión:

El análisis estadístico básico y los modelos estimados confirman la existencia de retornos crecientes a la educación, diferencias salariales por zona geográfica, jornada laboral y características demográficas. Además, se identificaron disparidades salariales por etnia.

Ejercicio 2: Considerando el data frame HousePrices del paquete AER que contiene los precios de los inmuebles vendidos en la ciudad de Windsor, Canadá, durante los meses de julio, agosto y septiembre de 1987:

a) Ajustar un modelo de regresión lineal múltiple para el logaritmo de los precios (price) en función del resto de las variables disponibles. Probar con un modelo que tenga a las variables tamaño del lote (lotsize), número de baños (bathrooms), número de habitaciones (bedrooms) y pisos (stories) en logaritmos y al resto en niveles. Comparar con otro modelo que tenga todas las variables en niveles. ¿Qué modelo es preferido? Para la comparación de los modelos, explorar el uso de la función stargazer() del paquete homónimo.

```
#inciso a

#Con los siguientes códigos exploramos las observaciones de la base de
datos:
str(HousePrices_1_)
summary(HousePrices_1_)

str(HousePrices_1_)

#Planteamos el primer modelo, antes procedemos a nombrar las variables
para una mejor interpretacion en la salida:

price <- HousePrices_1_$price
lotsize <- HousePrices_1_$lotsize
bedrooms <- HousePrices_1_$bedrooms
bathrooms <- HousePrices_1_$bathrooms
stories <- HousePrices_1_$stories
driveway <- HousePrices_1_$driveway
recreation <- HousePrices_1_$recreation
fullbase <- HousePrices_1_$fullbase
gasheat <- HousePrices_1_$gasheat
aircon <- HousePrices_1_$aircon
garage <- HousePrices_1_$garage
model1 <- lm(log(price) ~ log(lotsize) + log(bedrooms) + log(bathrooms)
+ log(stories) +
              driveway + recreation + fullbase + gasheat + aircon +
garage + prefer, data=HousePrices_1_)
summary(model1)

#Planteamos el segundo modelo, con todas las variables en niveles:
model2 <- lm(log(price) ~ lotsize + bedrooms + bathrooms + stories +
```

```

        driveway + recreation + fullbase + gasheat + aircon +
garage + prefer, data=HousePrices_1_)
summary(model2)

#En esta instancia procedemos a comprar los modelos, utilizando la
funcion que a continuacion procedemos a instalar:
#Comparacion de modelos:
install.packages("stargazer")
library(stargazer)

stargazer(model1, model2, type = "text", title = "Comparación de
Modelos", align = TRUE)
#Las estrellas indican significancia estadística: *** (muy
significativa), ** (moderada), * (débil), sin estrellas (no
significativa).

```

Salida:

```

> #inciso a
>
> #Con los siguientes códigos exploramos las observaciones de la base de
datos:
> str(HousePrices_1_)
Classes 'spec_tbl_df', 'tbl_df', 'tbl' and 'data.frame':  546 obs. of
12 variables:
 $ price      : num  42000 38500 49500 60500 61000 66000 66000 69000
83800 88500 ...
 $ lotsize    : num  5850 4000 3060 6650 6360 4160 3880 4160 4800 5500
...
 $ bedrooms   : num  3 2 3 3 2 3 3 3 3 3 ...
 $ bathrooms  : num  1 1 1 1 1 1 2 1 1 2 ...
 $ stories    : num  2 1 1 2 1 1 2 3 1 4 ...
 $ driveway   : chr   "yes" "yes" "yes" "yes" ...
 $ recreation: chr   "no" "no" "no" "yes" ...
 $ fullbase   : chr   "yes" "no" "no" "no" ...
 $ gasheat    : chr   "no" "no" "no" "no" ...
 $ aircon     : chr   "no" "no" "no" "no" ...
 $ garage     : num  1 0 0 0 0 0 2 0 0 1 ...
 $ prefer     : chr   "no" "no" "no" "no" ...
- attr(*, "spec")=List of 3
 ..$ cols      :List of 12
 .. ..$ price    : list()
 .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ lotsize   : list()
 .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ bedrooms  : list()

```

```

.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ bathrooms : list()
.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ stories   : list()
.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ driveway  : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ recreation: list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ fullbase  : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ gasheat   : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ aircon    : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ garage    : list()
.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ prefer    : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
..$ default: list()
.. - attr(*, "class")= chr [1:2] "collector_guess" "collector"
..$ delim   : chr ","
.. - attr(*, "class")= chr "col_spec"
- attr(*, "problems")=<externalptr>
> summary(HousePrices_1_)
      price      lotsize      bedrooms      bathrooms
stories
Min.   : 25000  Min.    : 1650  Min.     :1.000  Min.     :1.000  Min.
1st Qu.: 49125  1st Qu.: 3600  1st Qu.:2.000  1st Qu.:1.000  1st
Qu.:1.000
Median : 62000  Median : 4600  Median :3.000  Median :1.000  Median
:2.000
Mean   : 68122  Mean    : 5150  Mean    :2.965  Mean    :1.286  Mean
:1.808
3rd Qu.: 82000  3rd Qu.: 6360  3rd Qu.:3.000  3rd Qu.:2.000  3rd
Qu.:2.000
Max.   :190000  Max.    :16200  Max.    :6.000  Max.    :4.000  Max.
:4.000
      driveway      recreation      fullbase      gasheat

```

```

aircon
  Length:546      Length:546      Length:546      Length:546
Length:546
  Class :character  Class :character  Class :character  Class
:character  Class :character
  Mode :character  Mode :character  Mode :character  Mode
:character  Mode :character

      garage      prefer
Min.   :0.0000   Length:546
1st Qu.:0.0000   Class :character
Median :0.0000   Mode :character
Mean   :0.6923
3rd Qu.:1.0000
Max.   :3.0000
>
> str(HousePrices_1_)
Classes 'spec_tbl_df', 'tbl_df', 'tbl' and 'data.frame':  546 obs. of
12 variables:
 $ price      : num  42000 38500 49500 60500 61000 66000 66000 69000
83800 88500 ...
 $ lotsize    : num  5850 4000 3060 6650 6360 4160 3880 4160 4800 5500
...
 $ bedrooms   : num  3 2 3 3 2 3 3 3 3 3 ...
 $ bathrooms  : num  1 1 1 1 1 1 2 1 1 2 ...
 $ stories    : num  2 1 1 2 1 1 2 3 1 4 ...
 $ driveway   : chr  "yes" "yes" "yes" "yes" ...
 $ recreation: chr  "no" "no" "no" "yes" ...
 $ fullbase   : chr  "yes" "no" "no" "no" ...
 $ gasheat    : chr  "no" "no" "no" "no" ...
 $ aircon     : chr  "no" "no" "no" "no" ...
 $ garage     : num  1 0 0 0 0 0 2 0 0 1 ...
 $ prefer     : chr  "no" "no" "no" "no" ...
- attr(*, "spec")=List of 3
 ..$ cols      :List of 12
 .. ..$ price    : list()
 .. .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ lotsize   : list()
 .. .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ bedrooms  : list()
 .. .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ bathrooms : list()
 .. .. ..- attr(*, "class")= chr [1:2] "collector_double" "collector"
 .. ..$ stories   : list()

```

```

.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ driveway : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ recreation: list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ fullbase : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ gasheat : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ aircon : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
.. ..$ garage : list()
.. .. - attr(*, "class")= chr [1:2] "collector_double" "collector"
.. ..$ prefer : list()
.. .. - attr(*, "class")= chr [1:2] "collector_character"
"collector"
..$ default: list()
.. - attr(*, "class")= chr [1:2] "collector_guess" "collector"
..$ delim : chr ","
.. - attr(*, "class")= chr "col_spec"
- attr(*, "problems")=<externalptr>
>
> #Planteamos el primer modelo, antes procedemos a nombrar las variables
para una mejor interpretacion en la salida:
>
>
> price <- HousePrices_1_$price
> lotsize <- HousePrices_1_$lotsize
> bedrooms <- HousePrices_1_$bedrooms
> bathrooms <- HousePrices_1_$bathrooms
> stories <- HousePrices_1_$stories
> driveway <- HousePrices_1_$driveway
> recreation <- HousePrices_1_$recreation
> fullbase <- HousePrices_1_$fullbase
> gasheat <- HousePrices_1_$gasheat
> aircon <- HousePrices_1_$aircon
> garage <- HousePrices_1_$garage
> model1 <- lm(log(price) ~ log(lotsize) + log(bedrooms) +
log(bathrooms) + log(stories) +
+ driveway + recreation + fullbase + gasheat + aircon +
garage + prefer, data=HousePrices_1_)

```

```

> summary(model1)

Call:
lm(formula = log(price) ~ log(lotsize) + log(bedrooms) + log(bathrooms)
+
  log(stories) + driveway + recreation + fullbase + gasheat +
  aircon + garage + prefer, data = HousePrices_1_)

Residuals:
    Min       1Q   Median       3Q      Max
-0.69885 -0.12005  0.01215  0.12755  0.67404

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    7.92048    0.21918  36.137 < 2e-16 ***
log(lotsize)    0.31292    0.02692  11.623 < 2e-16 ***
log(bedrooms)   0.08879    0.04373   2.031 0.042785 *
log(bathrooms)  0.26377    0.03122   8.450 2.78e-16 ***
log(stories)    0.16523    0.02493   6.627 8.41e-11 ***
drivewayyes     0.10954    0.02836   3.863 0.000126 ***
recreationyes   0.05970    0.02615   2.283 0.022824 *
fullbaseyes     0.09564    0.02166   4.417 1.21e-05 ***
gasheatyes      0.17335    0.04412   3.929 9.63e-05 ***
airconyes       0.17078    0.02127   8.030 6.24e-15 ***
garage          0.04892    0.01154   4.238 2.65e-05 ***
preferyes       0.12968    0.02278   5.693 2.07e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2113 on 534 degrees of freedom
Multiple R-squared:  0.6839, Adjusted R-squared:  0.6774
F-statistic: 105 on 11 and 534 DF, p-value: < 2.2e-16

>
> #Planteamos el segundo modelo, con todas las variables en niveles:
> model2 <- lm(log(price) ~ lotsize + bedrooms + bathrooms + stories +
+
+   driveway + recreation + fullbase + gasheat + aircon +
+   garage + prefer, data=HousePrices_1_)
> summary(model2)

Call:
lm(formula = log(price) ~ lotsize + bedrooms + bathrooms + stories +
  driveway + recreation + fullbase + gasheat + aircon + garage +
  prefer, data = HousePrices_1_)

Residuals:

```

```
      Min      1Q   Median      3Q      Max
-0.67865 -0.12211  0.01666  0.12868  0.67737
```

Coefficients:

```
      Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.003e+01  4.724e-02 212.210 < 2e-16 ***
lotsize      5.057e-05  4.854e-06  10.418 < 2e-16 ***
bedrooms     3.402e-02  1.451e-02   2.345  0.01939 *
bathrooms    1.678e-01  2.065e-02   8.126 3.10e-15 ***
stories      9.227e-02  1.282e-02   7.197 2.10e-12 ***
drivewayyes  1.307e-01  2.834e-02   4.610 5.04e-06 ***
recreationyes 7.352e-02  2.633e-02   2.792  0.00542 **
fullbaseyes  9.940e-02  2.200e-02   4.517 7.72e-06 ***
gasheatyes   1.784e-01  4.458e-02   4.000 7.22e-05 ***
airconyes    1.780e-01  2.155e-02   8.262 1.14e-15 ***
garage       5.076e-02  1.165e-02   4.358 1.58e-05 ***
preferyes    1.271e-01  2.313e-02   5.496 6.02e-08 ***
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2137 on 534 degrees of freedom

Multiple R-squared: 0.6766, Adjusted R-squared: 0.6699

F-statistic: 101.6 on 11 and 534 DF, p-value: < 2.2e-16

>

> #En esta intancia procedemos a comprar los modelos, utilizando la
funcion que a continuacion procedemos a instalar:

> #Comparacion de modelos:

> install.packages("stargazer")

Error in install.packages : Updating loaded packages

> library(stargazer)

>

> stargazer(model1, model2, type = "text", title = "Comparación de
Modelos", align = TRUE)

Comparación de Modelos

=====

Dependent variable:

log(price)

(1)

(2)

log(lotsize) 0.313***
(0.027)

log(bedrooms) 0.089**

	(0.044)	
log(bathrooms)	0.264*** (0.031)	
log(stories)	0.165*** (0.025)	
lotsize		0.0001*** (0.0000)
bedrooms		0.034** (0.015)
bathrooms		0.168*** (0.021)
stories		0.092*** (0.013)
drivewayyes	0.110*** (0.028)	0.131*** (0.028)
recreationyes	0.060** (0.026)	0.074*** (0.026)
fullbaseyes	0.096*** (0.022)	0.099*** (0.022)
gasheatyes	0.173*** (0.044)	0.178*** (0.045)
airconyes	0.171*** (0.021)	0.178*** (0.022)
garage	0.049*** (0.012)	0.051*** (0.012)
preferyes	0.130*** (0.023)	0.127*** (0.023)
Constant	7.920*** (0.219)	10.026*** (0.047)

Observations	546	546


```

R2                                0.684                0.677
Adjusted R2                       0.677                0.670
Residual Std. Error (df = 534)    0.211                0.214
F Statistic (df = 11; 534)        105.034***         101.560***
=====
Note:                             *p<0.1; **p<0.05; ***p<0.01
> #Las estrellas indican significancia estadística: *** (muy
significativa), ** (moderada), * (débil), sin estrellas (no
significativa).

```

Fundamentación econométrica para la elección de modelos de regresión:

Comparando adecuadamente ambos modelos de regresión y aplicando los conocimientos adquiridos en Econometría I, se prefiere el primer modelo formulado en logaritmos ya que presenta un mayor R^2 ajustado (0.677 frente a 0.670). El R^2 ajustado es una versión del R^2 que tiene en cuenta la cantidad de variables incluidas en el modelo. Es útil para comparar modelos con diferentes cantidades de variables y ayuda a elegir el más parsimonioso (el que explica bien con menos, pues penaliza la inclusión de variables irrelevantes en el modelo).

Adicionalmente los respectivos R^2 de cada modelo indican que porcentaje de los mismos explican la variabilidad en el logaritmo del precio de las viviendas. En este sentido, el modelo 1 explica el 68,4% de la variabilidad en el logaritmo del precio de las viviendas, mientras que el modelo 2 explica el 67,7% de la misma. Por lo tanto, se concluye que de ambas formas el modelo 1 ajusta mejor los datos.

La interpretación de cada variable se detalla a continuación:

- Log(lotsize): Un aumento del 1% en el tamaño del lote se asocia con un aumento del 0.367% en el precio, ceteris paribus. (Porque tanto la X como la Y están en log, y en modelos log-log como este, los coeficientes se interpretan como elasticidades)
- Bedrooms: Cada dormitorio adicional se asocia con un aumento del 5.2% en el precio (porque Y está en log, pero X no).
- Bathrooms: Cada baño adicional se asocia con un aumento del 12.8% en el precio.
- Constante: Es el valor esperado de log(price) cuando todas las variables son cero. En este caso no tiene interpretación económica útil.

b) Realizar un análisis gráfico y pruebas de diagnóstico apropiadas sobre los residuos del modelo preferido. Para algunos contrastes se recomienda explorar y aplicar funciones disponibles en el paquete lmtest.-

En este inciso se pide realizar un diagnóstico de modo que se pueda verificar si se cumplen los supuestos del modelo. Se esperaría verificar que **los residuos estén distribuidos aleatoriamente**, sin forma o patrón visible.

Script:

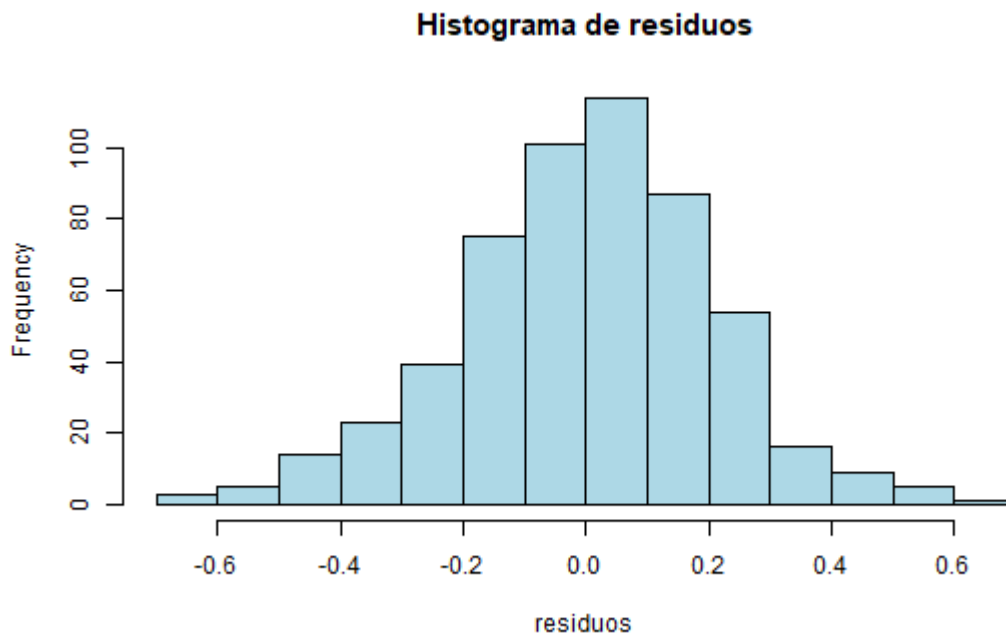
```
#inciso b
#Planteamos los residuos de la regresion:

residuos <- model1$residuals

#como no tiene ningun tipo de sentido correr el objeto "residuos" porque
se correrian como 3 hojas que no tienen interpretacion visible, elegimos
correr un grafico donde visualmente veremos la normalidad de los
residuos:

hist(residuos, main="Histograma de residuos", col="lightblue")
```

Consola:



```
#adicionalmente, una prueba mas formal para la normalidad antes
mencionada seria realizar la llamada "shapiro"
shapiro.test(residuos)
```

Consola:

```
> #adicionalmente, una prueba mas formal para la normalidad antes
mencionada seria realizar la llamada "shapiro"
> shapiro.test(residuos)
```

Shapiro-Wilk normality test

```
data:  residuos
W = 0.99266, p-value = 0.008857
```

Conclusión: Dado que el p-valor del test de Shapiro-Wilk es menor a 0.05 —nuestro nivel de significancia—, se rechaza la hipótesis nula de normalidad en los residuos. Si bien esto indica que los errores no siguen una distribución normal, en el contexto de regresión lineal esto no necesariamente invalida el modelo, ya que lo que se busca principalmente es aleatoriedad (ruido blanco) en los residuos, más que estricta normalidad. No obstante, si se requiere hacer inferencia (test t o F), podría ser recomendable considerar transformaciones o métodos robustos.

```
#Probamos ahora heterocedasticidad (Breusch-Pagan)
install.packages("lmtest")
library(lmtest)
bptest(model1)
```

Consola:

```
> #Probamos ahora heterocedasticidad (Breusch-Pagan)
> install.packages("lmtest")
> library(lmtest)
> bptest(model1)

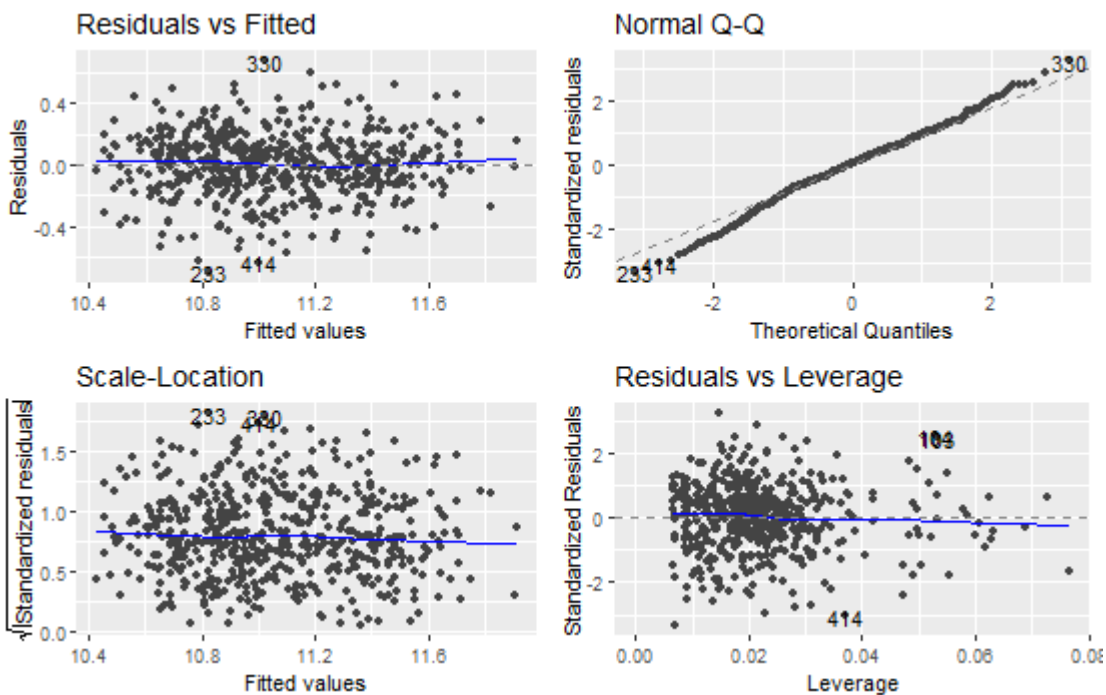
studentized Breusch-Pagan test

data:  model1
BP = 23.282, df = 11, p-value = 0.01612
```

Conclusión: siguiendo las hipótesis para la prueba Breusch-Pagan, Hipótesis nula H0: los residuos tienen varianza constante (no hay heterocedasticidad). Hipótesis alternativa H1: hay heterocedasticidad. Como el p-valor < 0.05, rechazamos H0, hay heterocedasticidad.

```
#A continuacion, en primera instancia procedimos a realizar graficos de
los residuos contra cada variable pero sinceramente le vimos poca
interpretacion y utilidad, por lo que (indagando al respecto) optamos
por realizar un grafico mas limpio como el que sigue:
# Diagnóstico gráfico completo del modelo
par(mfrow=c(2,2))
#para hacer una version moderna del grafico instalamos este paquete y
muy contentos con el resultado
```

```
install.packages("ggfortify")
library(ggfortify)
autoplot(model1)
```



Interpretación:

- **Residuals vs Fitted:** No hay un patrón curvo claro, buena linealidad.
- **Q-Q:** Los residuos se desvían en los extremos → ligera no-normalidad (consistente con tu Shapiro test).
- **Scale-Location:** Dispersión bastante constante, leve heterocedasticidad (ya lo sabíamos por el BP test).
- **Residuals vs Leverage:** Algunos puntos influyentes (4190, 4730) que convendría revisar.

c) Sobre la base del modelo preferido en a), ¿cuál es el precio esperado en logaritmos para una casa de dos pisos, 4700 pies cuadrados de superficie, tres habitaciones, dos baños, entrada para auto, sin cuarto de entretenimiento, sótano completo, sin calefacción a gas o aire acondicionado y garage para dos autos, que no esté localizada en un área de preferencia? Reportar también el intervalo de predicción.

```
HousePrices_lm <-
lm(log(price)~log(lotsize)+log(bedrooms)+log(bathrooms)+log(stories)+driveaway+recreation+fullbase+gasheat+aircon+garage+prefer,data=HousePrices)

predict(HousePrices_lm, newdata = data.frame(lotsize = 4700, bedrooms =
```

```
3, bathrooms = 2, stories = 2, driveway = "yes", recreation = "no",
fullbase = "yes", gasheat = "no", aircon = "no", garage = 2, prefer =
"no"), interval= "prediction")
```

Salida:

```
> predict(HousePrices_lm, newdata = data.frame(lotsize = 4700, bedrooms = 3,
bathrooms = 2, stories = 2, driveway = "yes", recreation = "no", fullbase =
"yes", gasheat = "no", aircon = "no", garage = 2, prefer = "no"), interval=
"prediction")
      fit      lwr      upr
1 11.26422 10.84492 11.68352
```

```
predict(HousePrices_lm, newdata = data.frame(lotsize = 4700, bedrooms = 3,
bathrooms = 2, stories = 2, driveway = "yes", recreation = "no", fullbase =
"yes", gasheat = "no", aircon = "no", garage = 2, prefer = "no"), interval=
"confidence")
```

Salida:

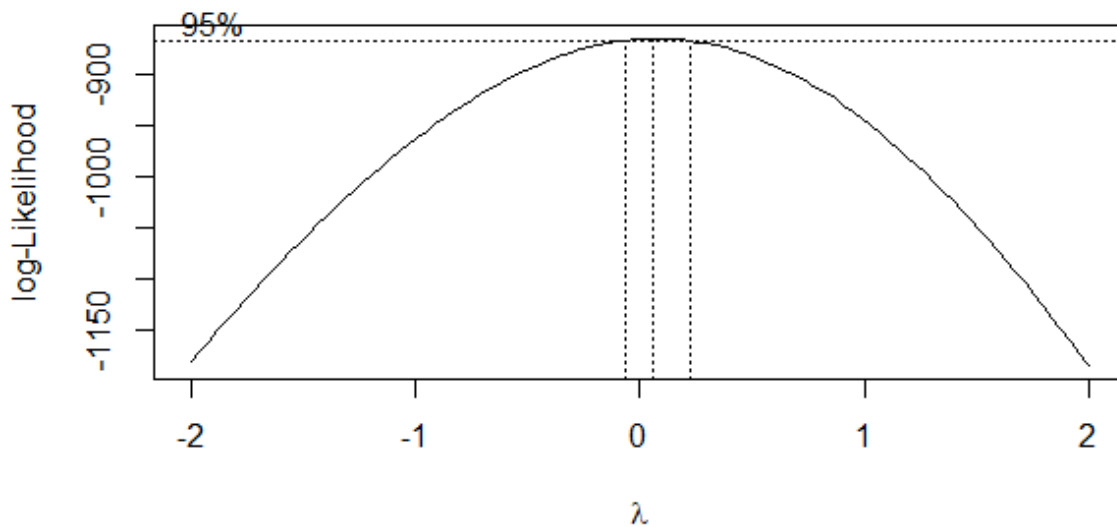
```
> predict(HousePrices_lm, newdata = data.frame(lotsize = 4700, bedrooms = 3,
bathrooms = 2, stories = 2, driveway = "yes", recreation = "no", fullbase =
"yes", gasheat = "no", aircon = "no", garage = 2, prefer = "no"), interval=
"confidence")
      fit      lwr      upr
1 11.26422 11.20461 11.32382
```

d) Para determinar si la transformación logarítmica de la variable dependiente es realmente apropiada, puede resultar útil probar otras transformaciones de Box-Cox. Para esto, explorar y utilizar la función `boxcox()` del paquete MASS. ¿Qué se puede concluir?

Para ver que transformación de la variable dependiente es más adecuada mediante la utilización de la función `Boxcox`, primero planteamos el modelo sin dicha variable en logaritmos:

```
Houseprices_lm_sin_log<- lm(price ~ log(lotsize) + log(bedrooms) +
log(bathrooms) + log(stories) + driveway + recreation + fullbase +
gasheat + aircon + garage + prefer, data = HousePrices)
```

```
install.packages("MASS")
library(MASS)
boxcox(Houseprices_lm_sin_log)
```



La transformación de Box-Cox se utiliza en modelos de regresión para encontrar la mejor forma de transformar la variable dependiente, explora una familia de transformaciones para la variable dependiente usando un parámetro λ . El valor óptimo de λ es aquel que maximiza la log-verosimilitud del modelo, es decir, el que genera el mejor ajuste de los datos. Si λ es cercano a cero, entonces la transformación logarítmica de la variable dependiente es apropiada.

Calculamos el lambda óptimo:

```
boxcox1<- boxcox(Houseprices_lm_sin_log)
lambda<- boxcox1$x [which.max(boxcox1$y)]
lambda
```

Salida:

```
> boxcox1<- boxcox(Houseprices_lm_sin_log)
> lambda<- boxcox1$x [which.max(boxcox1$y)]
> lambda
[1] 0.06060606
```

El valor óptimo de λ obtenido mediante el análisis de Box-Cox fue 0.06060606 , el cual es muy cercano a cero, y dado que el intervalo de confianza del 95% también contiene al cero, se concluye que la transformación logarítmica de la variable dependiente es adecuada .

Parte II: Estructuras de control y funciones.

1) Utilizando estructuras de control analizar la viabilidad de un portafolio de inversión mediante el cálculo del Valor Actual Neto (VAN). Para ello, considerar una inversión inicial y una serie de flujos de caja futuros. Aplicar una tasa de interés para descontar los flujos de caja y determinar el VAN del proyecto. El programa deberá imprimir si la inversión es viable, indicando si el VAN es positivo (proyecto recomendable), igual a cero (proyecto indiferente) o negativo (proyecto no recomendable).

```
#Estructuras de control y funciones
#1)
inversion_inicial <- 100000
flujos_caja_futuros <- c(25000, 30000, 35000, 40000, 45000, 50000,
55000, 60000, 65000, 70000)
tasa_interes <- 0.10

VAN<- -inversion_inicial

for(i in 1:length(flujos_caja_futuros)) {
  VAN <- VAN + flujos_caja_futuros[i] / (1 + tasa_interes)^i
}
print(VAN)

if (VAN > 0) {
  print("Proyecto recomendable")
} else if (VAN == 0) {
  print("Proyecto indiferente")
} else {
  print("Proyecto no recomendable")
}
```

Salida:

```

> inversion_inicial <- 100000
> flujos_caja_futuros <- c(25000, 30000, 35000, 40000, 45000, 50000, 55000,
60000, 65000, 70000)
> tasa_interes <- 0.10
>
> VAN<- -inversion_inicial
>
> for(i in 1:length(flujos_caja_futuros)) {
+   VAN <- VAN + flujos_caja_futuros[i] / (1 + tasa_interes)^i
+ }
> print(VAN)
[1] 168070.9
>
> if (VAN > 0) {
+   print("Proyecto recomendable")
+ } else if (VAN == 0) {
+   print("Proyecto indiferente")
+ } else {
+   print("Proyecto no recomendable")
+ }
[1] "Proyecto recomendable"

```

2) Implementar una función que calcule la elasticidad precio de la demanda a partir de dos precios y sus respectivas cantidades demandadas. La función debe recibir como parámetros los valores de precio y cantidad antes y después del cambio, calcular la elasticidad utilizando la fórmula correspondiente y devolver tanto el valor de la elasticidad como una clasificación del bien.

```

elasticidad_precio_demanda <- function(P1, P2, Q1, Q2) {
  elasticidad <- ((Q2 - Q1) / Q1) / ((P2 - P1) / P1)
  message("La elasticidad precio de la demanda es: ", elasticidad)
  if (abs(elasticidad) > 1) {
    print(" Bien Elástico")
  } else if (abs(elasticidad) == 1) {
    print("Bien con Elasticidad unitaria")
  } else {
    print("Bien Inelástico")
  }
}
elasticidad_precio_demanda(10, 12, 200, 135)

```

Salida:

```

> elasticidad_precio_demanda <- function(P1, P2, Q1, Q2) {

```



```

+ elasticidad <- ((Q2 - Q1) / Q1) / ((P2 - P1) / P1)
+ message("La elasticidad precio de la demanda es: ", elasticidad)
+ if (abs(elasticidad) > 1) {
+   print(" Bien Elástico")
+ } else if (abs(elasticidad) == 1) {
+   print("Bien con Elasticidad unitaria")
+ } else {
+   print("Bien Inelástico")
+ }
+ }
> elasticidad_precio_demanda(10, 12, 200, 135)
La elasticidad precio de la demanda es: -1.625
[1] " Bien Elástico"

```

3) Opcional: resolver todo lo anterior con Python.

Entrada:

```

def calcular_van(inversion_inicial, flujos, tasa):
    van = -inversion_inicial
    for periodo, flujo in enumerate(flujos, start=1):
        van += flujo / ((1 + tasa) ** periodo)
    return van

def evaluar_proyecto(van):
    if van > 0:
        print("Proyecto recomendable (VAN positivo).")
    elif van == 0:
        print("Proyecto indiferente (VAN = 0).")
    else:
        print("Proyecto no recomendable (VAN negativo).")

if __name__ == "__main__":
    inversion_inicial = 100_000
    flujos = [25_000, 30_000, 35_000, 40_000, 45_000,
              50_000, 55_000, 60_000, 65_000, 70_000]
    tasa = 0.10

    van = calcular_van(inversion_inicial, flujos, tasa)
    print(f"VAN calculado: {van:,.2f}")
    evaluar_proyecto(van)

```

Salida:

```
VAN calculado: 168,070.89
Proyecto recomendable (VAN positivo).
```

Entrada:

```
def elasticidad_precio_demanda(P1, P2, Q1, Q2):
    elasticidad = ((Q2 - Q1) / Q1) / ((P2 - P1) / P1)
    print(f"La elasticidad precio de la demanda es: {elasticidad}")
    if abs(elasticidad) > 1:
        print("Bien Elástico")
    elif abs(elasticidad) == 1:
        print("Bien con Elasticidad unitaria")
    else:
        print("Bien Inelástico")

elasticidad_precio_demanda(10, 12, 200, 135)
```

Salida:

```
La elasticidad precio de la demanda es: -1.625
Bien Elástico
```

UNIDAD 3:

Parte I: Optimización

Utilizando los datos de la tabla 12.1 de la pág. 500 del libro de Judge, G. G., Hill, R. C., Griffiths, W. E., Luetkepohl, H. y Lee, T. C. (1988), Introduction to the Theory and Practice of Econometrics, Segunda Edición, Wiley, New York:

a) Graficar la función de la suma de cuadrados de residuos del modelo planteado en el texto. Esta función es un polinomio de cuarto grado en el parámetro beta.

función:

$$S(\beta) = \sum_{t=1}^T (Y_t - \beta X_{t1} - \beta^2 X_{t2})^2$$

Script:

```
#nombramos los valores de las tablas con sus respectivas variables:
Yt <- datosJudge1988[,2]; Yt
Xt1 <- datosJudge1988[,3]; Xt1
Xt2 <- datosJudge1988[,4]; Xt2
# funcion de la suma de cuadrados:
S_beta <- function(beta, Yt, Xt1, Xt2){
  sum((Yt - beta * Xt1 - beta^2 * Xt2)^2)}
S_beta
#procedemos a evaluar a la funcion con los valores beta (una secuencia):
beta_vals <- seq(-5, 5, by=0.01)
```

```
S_β_vals <- sapply(beta_vals, S_β, Yt = Yt, Xt1 = Xt1, Xt2 = Xt2)
#graficamos:
plot(beta_vals, S_β_vals, type="l", col="darkred", lwd=2, main="GRAFICO
SCR DEL MODELO", xlab= expression(beta), ylab="S(beta)")
```

consola:

```
> #nombramos los valores de las tablas con sus respectivas variables:
```

```
> Yt <- datosJudge1988[,2]; Yt
```

```
# A tibble: 20 × 1
```

```
      y
  <dbl>
1  3.28
2  3.15
3  2.88
4 -0.467
5  1.21
6  1.39
7  1.14
8  2.32
9  0.998
10 0.379
11 1.11
12 0.428
13 0.011
14 1.18
15 1.86
16 0.388
17 0.651
18 0.593
19 0.046
20 1.15
```

```
> #nombramos los valores de las tablas con sus respectivas variables:
```

```
> Yt <- datosJudge1988[,2]; Yt
```

```
# A tibble: 20 × 1
```

```
      y
  <dbl>
1  3.28
2  3.15
3  2.88
4 -0.467
5  1.21
6  1.39
7  1.14
8  2.32
9  0.998
10 0.379
11 1.11
12 0.428
13 0.011
14 1.18
15 1.86
16 0.388
17 0.651
18 0.593
19 0.046
20 1.15
```

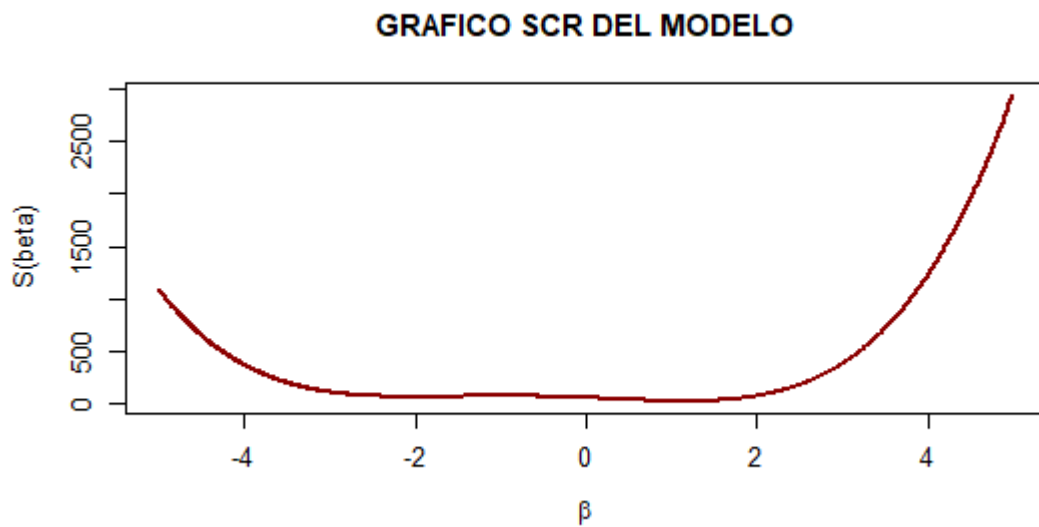
```
> Xt1 <- datosJudge1988[,3]; Xt1
```

```

# A tibble: 20 × 1
  x1
<dbl>
1 0.286
2 0.973
3 0.384
4 0.276
5 0.937
6 0.543
7 0.957
8 0.948
9 0.543
10 0.797
11 0.936
12 0.889
13 0.006
14 0.828
15 0.399
16 0.617
17 0.939
18 0.784
19 0.072
20 0.889
> Xt2 <- datosJudge1988[,4]; Xt2
# A tibble: 20 × 1
  x2
<dbl>
1 0.645
2 0.585
3 0.31
4 0.058
5 0.455
6 0.779
7 0.259
8 0.202
9 0.028
10 0.099
11 0.142
12 0.296
13 0.175
14 0.18
15 0.842
16 0.039
17 0.103
18 0.62
19 0.158
20 0.704
> # funcion de la suma de cuadrados:
> S_β <- function(beta, Yt, Xt1, Xt2){
+   sum((Yt-beta * Xt1- beta^2 * Xt2)^2)}
> S_β

```

```
function(beta, Yt, Xt1, Xt2){  
  sum((Yt-beta * Xt1- beta^2 * Xt2)^2)}  
> #procedemos a evaluar a la funcion con los valores beta (una secuencia):  
> beta_vals <- seq(-5, 5, by=0.01)  
> S_β_vals <- sapply(beta_vals, S_β, Yt = Yt, Xt1 = Xt1, Xt2 = Xt2)  
> #graficamos:  
> plot(beta_vals, S_β_vals, type="l", col="darkred", lwd=2, main="GRAFICO SCR  
DEL MODELO", xlab= expression(beta), ylab="S(beta)")
```



b) Calcular las raíces del polinomio de tercer grado que se obtiene al calcular la primera derivada de esa función con respecto a beta. Para esto explorar y utilizar la función polyroot de R.

$$2(\beta^3) * \sum_{t=1, T} (X_{t2})^2 + 3(\beta)^2 \sum_{t=1, T} X_{t1} * X_{t2} + \beta * (\sum_{t=1, T} (X_{t1})^2 - 2 \sum_{t=1, T} X_{t2} * Y_t) - \sum_{t=1, T} X_{t1} * Y_t = 0$$

script:

```
# Coeficientes del polinomio de la derivada
a3 <- 2 * sum(Xt2^2)
a2 <- 3 * sum(Xt1 * Xt2)
a1 <- sum(Xt1^2) - 2 * sum(Xt2 * Yt)
a0 <- -sum(Xt1 * Yt)

# Mostrar coeficientes
coeffs <- c(a3, a2, a1, a0)
print(coeffs)

# Calcular raíces, la función utilizada es polyroot() encuentra las raíces de un
polinomio dado su vector de coeficientes.
roots <- polyroot(coeffs)
print(roots)
```

consola:

```

> # Coeficientes del polinomio de la derivada
> a3 <- 2 * sum(Xt2^2)
> a2 <- 3 * sum(Xt1 * Xt2)
> a1 <- sum(Xt1^2) - 2 * sum(Xt2 * Yt)
> a0 <- -sum(Xt1 * Yt)
> # Mostrar coeficientes
> coeffs <- c(a3, a2, a1, a0)
> print(coeffs)
[1] 7.173858 13.193151 -10.885543 -16.484426
> # Calcular raíces, la función utilizada es polyroot() encuentra las raíces de
un polinomio dado su vector de coeficientes.
> roots <- polyroot(coeffs)
> print(roots)
[1] -0.4929270-4.276524e-22i -1.0270461+3.065830e-22i 0.8596199+1.210694e-22i

```

c) Calcular los puntos de inflexión del polinomio de cuarto grado.

```

# Despues de calcular las respectivas derivadas de la función,
calculamos los coeficientes de la tercera derivada
c1 <- 12 * sum(Xt2^2)
c0 <- 6 * sum(Xt1 * Xt2)
# La raíz de  $f'''(\beta) = 0$  es, por lo tanto su punto de inflexión
beta_inflexion <- -c0 / c1
print(beta_inflexion)

```

consola:

```

> # Después de calcular las respectivas derivadas de la función,
calculamos los coeficientes de la tercera derivada
> c1 <- 12 * sum(Xt2^2) ; c1
[1] 43.04315
> c0 <- 6 * sum(Xt1 * Xt2) ; c0
[1] 26.3863
> # La raíz de  $f'''(\beta) = 0$  es, por lo tanto su punto de inflexión
> beta_inflexion <- -c0 / c1
> # La raíz de  $f'''(\beta) = 0$  es, por lo tanto su punto de inflexión
> beta_inflexion <- -c0 / c1
> print(beta_inflexion)
[1] -0.6130198

```

Parte II: Simulación

1) Simular un modelo lineal para analizar el impacto de la heteroscedasticidad en una regresión lineal simple. Para ello, generar 10 mil observaciones de una variable dependiente

Y a partir de una relación lineal con una variable explicativa X, incorporando un término de error cuya varianza dependa de X. Comenzar la simulación con una semilla que garantice la reproducibilidad de los resultados.

Luego, estimar el modelo por Mínimos Cuadrados Ordinarios (OLS), realizar un contraste apropiado de heteroscedasticidad para verificar la presencia del problema y comparar las estimaciones obtenidas mediante OLS con las que se obtienen utilizando errores estándar robustos con la función `vcovHC` del paquete `sandwich` de R. Probar con los distintos métodos de estimación que permite la función `vcovHC`. Prestar especial atención a cómo la heteroscedasticidad afecta la inferencia estadística: ¿Se modifican los coeficientes estimados? ¿Cómo cambian los errores estándar de los coeficientes?

```
#Unidad 3- parte 2: Simulación

set.seed(123)

x<-rnorm(10000)

mu<-rnorm(10000,0, 1*(x)^2 )
y<- 0.5+2*x+mu

fit1<-lm(y~x)
summary(fit1)

#contrastes de heterocedasticidad:
##Test de Breusch-Pagan
library(lmtest)
bptest(fit1)
```

Salida:

```

> set.seed(123)
> x<-rnorm(10000)
> mu<-rnorm(10000,0, 1*(x)^2 )
> y<- 0.5+2*x+mu
>
> fit1<-lm(y~x)
> summary(fit1)

Call:
lm(formula = y ~ x)

Residuals:
    Min       1Q   Median       3Q      Max
-17.4190  -0.2477  -0.0067   0.2150  21.8484

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.50656    0.01689   29.99  <2e-16 ***
x            1.99359    0.01692  117.85  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.689 on 9998 degrees of freedom
Multiple R-squared:  0.5815, Adjusted R-squared:  0.5814
F-statistic: 1.389e+04 on 1 and 9998 DF, p-value: < 2.2e-16

>
> #contrastes de heterocedasticidad:
> ##Test de Breusch-Pagan
> library(lmtest)
> bptest(fit1)

            studentized Breusch-Pagan test

data:  fit1
BP = 4.5949, df = 1, p-value = 0.03207

```

Para un $\alpha=0,05$ se rechaza la hipótesis nula de homocedasticidad y se confirma la existencia de heterocedasticidad.

```

for (i in 1:length(tipos_vcovHC)) {
  cat(" Tipo:", tipos_vcovHC[i], " \n")
  print(coeftest(fit1, vcov. = vcovHC(fit1, type = tipos_vcovHC[i])))
  cat("\n")
}

```

Salida:

```
> for (i in 1:length(tipos_vcovHC)) {  
+   cat(" Tipo:", tipos_vcovHC[i], " \n")  
+   print(coeftest(fit1, vcov. = vcovHC(fit1, type = tipos_vcovHC[i])))  
+   cat("\n")  
+ }  
Tipo: HC3
```

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016905	29.966	< 2.2e-16 ***
x	1.993591	0.037253	53.515	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: const

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016892	29.988	< 2.2e-16 ***
x	1.993591	0.016916	117.854	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016895	29.983	< 2.2e-16 ***
x	1.993591	0.037224	53.556	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC0

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016895	29.983	< 2.2e-16 ***
x	1.993591	0.037224	53.556	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC1

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016897	29.980	< 2.2e-16 ***
x	1.993591	0.037228	53.551	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC2

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016900	29.974	< 2.2e-16 ***
x	1.993591	0.037239	53.535	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC4

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016910	29.956	< 2.2e-16 ***
x	1.993591	0.037275	53.484	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC4m

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016907	29.961	< 2.2e-16 ***
x	1.993591	0.037260	53.505	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tipo: HC5

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.506561	0.016903	29.968	< 2.2e-16 ***

```
x          1.993591    0.037254  53.513 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

2) Opcional: resolver el ejercicio anterior con Python.

```
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.diagnostic import het_breuschpagan

rng = np.random.RandomState(123)

x = rng.normal(size=10_000)
mu = rng.normal(loc=0, scale=x**2)
y = 0.5 + 2 * x + mu

X = sm.add_constant(x)
ols = sm.OLS(y, X).fit()
print(ols.summary())

#Test de BP
bp = het_breuschpagan(ols.resid, sm.add_constant(ols.fittedvalues))
lm_stat, lm_pvalue, f_stat, f_pvalue = map(float, bp)
print(f'\nBreusch-Pagan LM stat: {lm_stat:.4f}, p-value: {lm_pvalue:.4g}')
print(f'Breusch-Pagan F stat: {f_stat:.4f}, p-value: {f_pvalue:.4g}')

for cov in ['HC0', 'HC1', 'HC2', 'HC3', 'HC4', 'HC4m', 'HC5']:
    try:
        rob = ols.get_robustcov_results(cov_type=cov)
        print(f'\n{cov}')
        print(rob.summary().tables[1])
    except ValueError:
        pass
```

Salida:

```

                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.563
Model:                  OLS    Adj. R-squared:            0.563
Method:                 Least Squares    F-statistic:        1.290e+04
Date:                  Tue, 15 Apr 2025    Prob (F-statistic):    0.00
Time:                  04:50:51    Log-Likelihood:      -19572.
```

No. Observations: 10000 AIC: 3.915e+04
Df Residuals: 9998 BIC: 3.916e+04
Df Model: 1
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
-----	-----	-----	-----	-----	-----	-----
const	0.5185	0.017	30.262	0.000	0.485	0.552
x1	1.9496	0.017	113.582	0.000	1.916	1.983

Omnibus: 3043.840 Durbin-Watson: 1.950
Prob(Omnibus): 0.000 Jarque-Bera (JB): 306942.352
Skew: -0.441 Prob(JB): 0.00
Kurtosis: 30.127 Cond. No. 1.01

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Breusch-Pagan LM stat: 31.9373, p-value: 1.592e-08

Breusch-Pagan F stat: 32.0332, p-value: 1.557e-08

HC0

	coef	std err	t	P> t	[0.025	0.975]
-----	-----	-----	-----	-----	-----	-----
const	0.5185	0.017	30.350	0.000	0.485	0.552
x1	1.9496	0.038	50.888	0.000	1.875	2.025

HC1

	coef	std err	t	P> t	[0.025	0.975]
-----	-----	-----	-----	-----	-----	-----
const	0.5185	0.017	30.347	0.000	0.485	0.552
x1	1.9496	0.038	50.883	0.000	1.875	2.025

HC2

	coef	std err	t	P> t	[0.025	0.975]
-----	-----	-----	-----	-----	-----	-----
const	0.5185	0.017	30.341	0.000	0.485	0.552
x1	1.9496	0.038	50.868	0.000	1.875	2.025

HC3

	<i>coef</i>	<i>std err</i>	<i>t</i>	<i>P> t </i>	<i>[0.025</i>	<i>0.975]</i>
<i>const</i>	0.5185	0.017	30.332	0.000	0.485	0.552
<i>x1</i>	1.9496	0.038	50.848	0.000	1.874	2.025

Observamos que los valores numéricos difieren ligeramente entre R y Python dado que cada lenguaje usa su propia secuencia de números aleatorios aunque se fije la misma semilla (seed 123) pero aun así los resultados son los mismos.

Notamos también que en el caso del estadístico Breusch-Pagan se observan valores diferentes dado que R está usando la versión studentizada, mientras que statsmodels en Python reporta la clásica, representando ambas lo mismo y dando veredictos en la misma dirección pero cambiando la escala del estadístico.